



Which Scalability Approach is Right for our App?

In this lesson, you will learn which type of scaling is best for a given scenario.

We'll cover the following



- Pros and cons of vertical and horizontal scaling
- What about the code? Why does the code need to change when it has to run on multiple machines?
- Which scalability approach is right for our app?

Pros and cons of vertical and horizontal scaling

This is where I talk about the pluses and minuses of both scaling approaches.

Vertical scaling, as we learned before, is simpler in comparison to *horizontal scaling* because we do not have to touch the code or make any complex system configurations. It takes much less administrative, monitoring, and management efforts than managing a distributed environment when scaling horizontally.

A significant downside of vertical scaling is the availability risk. The server [?] are powerful but few in number. There is always a risk of them going down ^{Tr} and the entire website going offline, which doesn't happen when the system is scaled horizontally. In this scenario, the system is more *highly available*. 

What about the code? Why does the code need to change when it has to run on multiple machines?



If you intend to run the code in a distributed environment, it needs to be *stateless*. There should be no *state* in the code. What do I mean by this?

There should be no *static* instances in the *class*. *Static* instances hold application data and when a particular server goes down, all the static data/state is lost. The app is left in an inconsistent state.

In *object-oriented programming*, the *instance variables* hold object *state* in them. *Static variables* moreover hold *state* that spans across multiple *objects*. They generally hold *state* per *classloader*. Now, if the *server instance* running that *classloader* goes down, all the data is lost.

Also, whatever data static variables hold, it's not application-wide. For this reason, distributed memory like *Redis*, *Memcache*, etc., are used to maintain a consistent state application-wide. When writing applications for distributed systems, it's a good practice to avoid using *static* instances in the class. The state is typically persisted in a distributed memory store; this facilitates components to be *stateless*.

This is why *functional programming* got popular with distributed systems. The functions don't retain any state. However, the same behavior can also be achieved with prominent *OOP* languages.

I've delved into the concept (making code changes in the application layer when scaling horizontally) in much detail in the distributed transactions chapter of [my system design course](#). [Check it out](#).



Which scalability approach is right for our app?

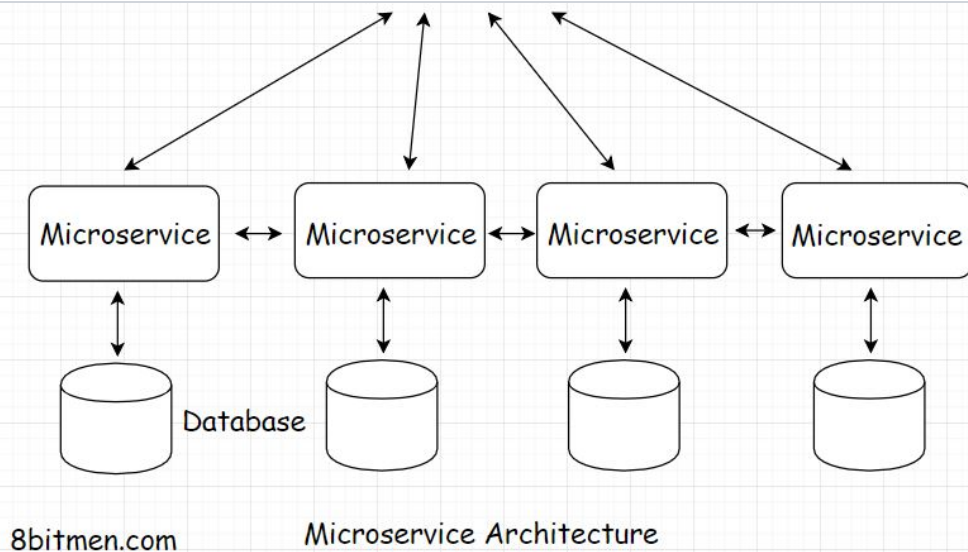


Always have a ballpark estimate in mind when designing your app. How much traffic will it have to deal with?

Today, development teams are adopting a distributed *microservices* architecture right from the start, and workloads (applications) are meant to be deployed on the cloud. So, inherently the workloads are horizontally scaled out on the fly.



 educative



The upsides of horizontal scaling include no limit to augmenting the hardware capacity. Data is replicated across different geographical regions as nodes and data centers are set up across the globe.

If our app is a utility or tool expected to receive minimal predictable traffic? For instance, an internal tool of an organization or something similar that is not mission-critical. We need not bother hosting it in a distributed environment. A single server is enough to manage the traffic. We can go ahead with vertical scaling when we are certain that the traffic load will not spike significantly in the near future.

If our app is a public-facing social app like a social network, a fitness app, an online game, or something similar, where the traffic is unpredictable. Both *high availability* and *horizontal scalability* are important to us.



Build these apps to deploy them on the cloud, and always have *horizontal scalability* in mind right from the start.

[← Back](#)[Mark As Completed](#)[Next →](#)[Types of Scalability](#)[Primary Bottlenecks That Hurt the Scal...](#)